Cover Page

TourPlan AI: Smart Travel & Trip Planning System

Software Engineering Lab Project Report

MBA Tech, Computer Engineering

[Student name(s)] | [Roll no(s)]

Guided by: [Faculty mentor]

[Institute / department name]

[Submission date]

Certificate

This is to certify that the project entitled "TourPlan AI: Smart Travel & Trip Planning System" is a bonafide work carried out by the student(s) named below in partial fulfilment of the Software Engineering Lab requirement.

Student(s): [Student name(s)], [Roll no(s)]

Faculty signature: _____ (to be signed by the faculty)

Head of Department signature: _____ (to be signed)

Declaration

I/We declare that this work is our own, that sources are acknowledged in the references, and that no results in this report have been invented. Sections marked as placeholders must be completed by the student.

Name and signature: [Student name(s)] _____ Date: [Submission date]

Acknowledgement

We thank our faculty mentor, the department and everyone who gave feedback on this project. We also acknowledge the open-source communities behind FastAPI, React, Leaflet, OpenStreetMap and the other tools listed in the references.

Edit this paragraph to add personal thanks.

Abstract

TourPlan AI is a web application that helps a traveller turn a rough idea (for example, 'three days near Mumbai under Rs 20,000') into a ranked destination list, a day-by-day itinerary, an itemised budget estimate, a map and a PDF. A deterministic engine produces every number and place from a sample dataset; an optional large language model (Gemini, configured only on the server) is limited to explaining the plan, and a rule-based fallback is used and labelled whenever the model is unavailable. The system uses React, FastAPI, SQLAlchemy and SQLite. This report documents the requirements, design, implementation and test approach. Test results are not stated here because the tests were not executed in the authoring environment; section 25 is a placeholder for real output.

Table of Contents

Cover Page 1

Certificate 2

Declaration 3

Acknowledgement 4

Abstract 5

Table of Contents 6

1. Introduction 8

2. Problem Statement 9

3. Existing System 10

4. Proposed System 11

5. Objectives and Scope 12

6. Functional Requirements 13

7. Non-Functional Requirements 14

8. Feasibility Study 15

9. Technology Stack 16

10. System Architecture 17

11. Use Case Design 18

12. Data Flow Diagrams 20

13. ER Diagram and Database Schema 22

14. AI Architecture 23

15. User Interface Design 24

16. Screenshots 25

17. Module Description 26

18. API Design 27

19. Budget Methodology 28

20. Security Design 29

21. Implementation Details 30

22. Testing Methodology 31

23. Test Cases 32

24. Validation and Error Handling 34

25. Actual Test Results 35

26. Limitations 36

27. Future Enhancements 37
28. Conclusion 38
29. References 39
30. Appendix A: Setup Commands 40
31. Appendix B: Sample Data Notice 41

1. Introduction

Trip planning normally means juggling search engines, maps, spreadsheets and chat apps. TourPlan AI brings destination discovery, itinerary building, budgeting, maps and an assistant into one application, and is built as a Software Engineering Lab project to practise requirement analysis, design, implementation and testing.

2. Problem Statement

Travellers struggle to decide where to go within a budget, to estimate total cost, and to arrange a realistic daily schedule. Generic chatbots can invent prices and places. The problem is to build a planner that is transparent about how it recommends, keeps all figures reproducible, and is honest about the limits of its data.

3. Existing System

• Booking portals show live prices but do not plan a whole trip or explain recommendations.

• Spreadsheets give control but need manual research and arithmetic.

• General chatbots are conversational but can hallucinate prices, places and distances, and give no editable plan.

4. Proposed System

A planner whose recommendation score and budget formulas are visible, whose places come from a database, and whose LLM is optional and constrained to explanation. Plans are saved, editable, mappable and exportable.

5. Objectives and Scope

• Rank destinations transparently (100-point score with reasons).

• Generate and edit itineraries.

• Estimate and track budget with over-budget warnings.

• Provide map, packing list, PDF export and an assistant.

• Keep secrets on the server, validate inputs, protect user data.

Out of scope: booking, live prices, payments, real-time availability, multi-currency trips, mobile native apps.

6. Functional Requirements

ID	Requirement
FR-01	Register with name, email and password (min 8 chars); duplicate emails rejected; passwords stored as bcrypt hashes.
FR-02	Log in and receive a signed JWT; log out clears the token; protected routes require it.
FR-03	Store a profile with home city, travel style and interests; use them as defaults.
FR-04	Browse 12 seeded destinations; search by name, country or region; filter by category, interest, climate, month and daily cost; sort.
FR-05	View a destination: description, practical information, best months, indicative costs, attractions with fees and durations, map pins.
FR-06	Save and remove favourite destinations (per user, idempotent).
FR-07	Generate a trip from a form (start city, destination or 'suggest', dates, travellers, budget, style, pace, transport, accommodation, interests).
FR-08	Rank destinations with a transparent 100-point score and show the reason for each component.
FR-09	Produce a day-by-day itinerary (morning/afternoon/evening) from the attraction dataset, with durations and fees.
FR-10	Estimate transport, accommodation, food, activities, local transport, shopping/misc and an emergency buffer; show total, per person and per day.
FR-11	Warn when the estimate exceeds the budget and propose cheaper alternatives; never silently exceed the budget.
FR-12	Save a generated or manual trip; list, search and filter trips by status; open, edit details, duplicate and delete (with confirmation).
FR-13	Edit an itinerary: change fields, add, remove and reorder activities; persist changes.
FR-14	Edit budget categories and actual spend; recompute totals immediately; show donut, daily and estimate-versus-actual charts.
FR-15	Show trip stops on an OpenStreetMap map with a route line, with map/list switching and a tile-failure notice.
FR-16	Packing checklist generated from climate, destination type and interests; items can be ticked, added and removed; progress shown.
FR-17	Export a trip itinerary and budget as a PDF; copy a plain-text trip summary.
FR-18	AI assistant on every page: extracts requirements from natural language, asks for missing details, recommends, plans, and refines ('cheaper', 'more adventure').
FR-19	Use a configured LLM only to explain a plan grounded in dataset facts; otherwise a rule-based fallback, always labelled as such.
FR-20	Persist chat conversations; allow deleting a conversation.
FR-21	Optional live weather and currency conversion with clearly labelled fallbacks.
FR-22	Plane-flight page transition with a setting to disable it and automatic respect for reduced-motion preferences.

7. Non-Functional Requirements

ID	Requirement
NFR-01 Security	bcrypt password hashing; JWT (HS256) with expiry; per-user authorisation returning 404 for other users' resources; secrets only in backend environment variables; CORS allow-list.
NFR-02 Validation	All inputs validated by Pydantic (types, ranges, enums, date order, coordinate pairs); meaningful 4xx messages; generic 500 without internals.
NFR-03 Honesty	Prices are labelled as estimates from sample data; fallback AI output is labelled rule-based; the app never claims to book anything.
NFR-04 Reliability	External services (LLM, weather, FX, map tiles) are optional and fail gracefully; the core flow works fully offline from seeded data.
NFR-05 Performance	Single-process SQLite deployment suited to a demo; no heavy computation (scoring 12 destinations per request).
NFR-06 Usability	Responsive layout (mobile, tablet, desktop); loading skeletons, empty and error states; toast feedback; confirmation before destructive actions.
NFR-07 Accessibility	Labelled controls, keyboard operation, visible focus ring, aria roles for tabs and progress, skip link, reduced-motion support.
NFR-08 Maintainability	Modular backend (routers / services / domain engine / AI layer), typed frontend, pure functions for budget and travel logic.
NFR-09 Portability	Runs on Windows, macOS and Linux with Python 3.11+ and Node 18+; no external database server.
NFR-10 Testability	Pure-function budget and travel modules, API tests with an isolated SQLite file, mocked LLM responses.

8. Feasibility Study

• Technical: all tools are free and open source; runs on a laptop.

• Economic: no paid services required; the LLM key is optional.

• Operational: single-user demo deployment; simple setup via two commands.

• Schedule: modular design allowed backend, frontend and documents to progress in parallel.

9. Technology Stack

Layer	Technology	Why
Frontend framework	React 18 + Vite + TypeScript	Component model, fast dev server, static typing catches data-shape mistakes.
Styling	Tailwind CSS 3	Utility classes give a consistent glassmorphism design system quickly.
Motion	Framer Motion	Declarative animation for counters, card entrances, rings and the plane transition.
Routing	React Router 6	Real routes and back-button support.
Maps	React-Leaflet + OpenStreetMap	Free, open tiles; works without an API key.
Charts	Recharts	Donut and bar charts for the budget.
Backend	Python + FastAPI	Typed endpoints, automatic OpenAPI/Swagger documentation.
Validation	Pydantic v2	Declarative request validation and response models.
Database	SQLAlchemy 2 + SQLite	Relational schema with constraints; zero-setup persistence.
Security	bcrypt + PyJWT	Salted password hashing and signed, expiring tokens.
AI	Google Gemini REST API (optional) + rule-based fallback	Configurable provider; the app works without a key.
PDF	ReportLab	Server-side itinerary export.
Testing	pytest + FastAPI TestClient; Vitest	Backend API/unit tests; frontend helper tests.

10. System Architecture

The browser runs a React single-page app. It calls the FastAPI REST API with a JWT. The API reads and writes SQLite via SQLAlchemy, calls the deterministic engine, and optionally calls Gemini and public weather/currency services.

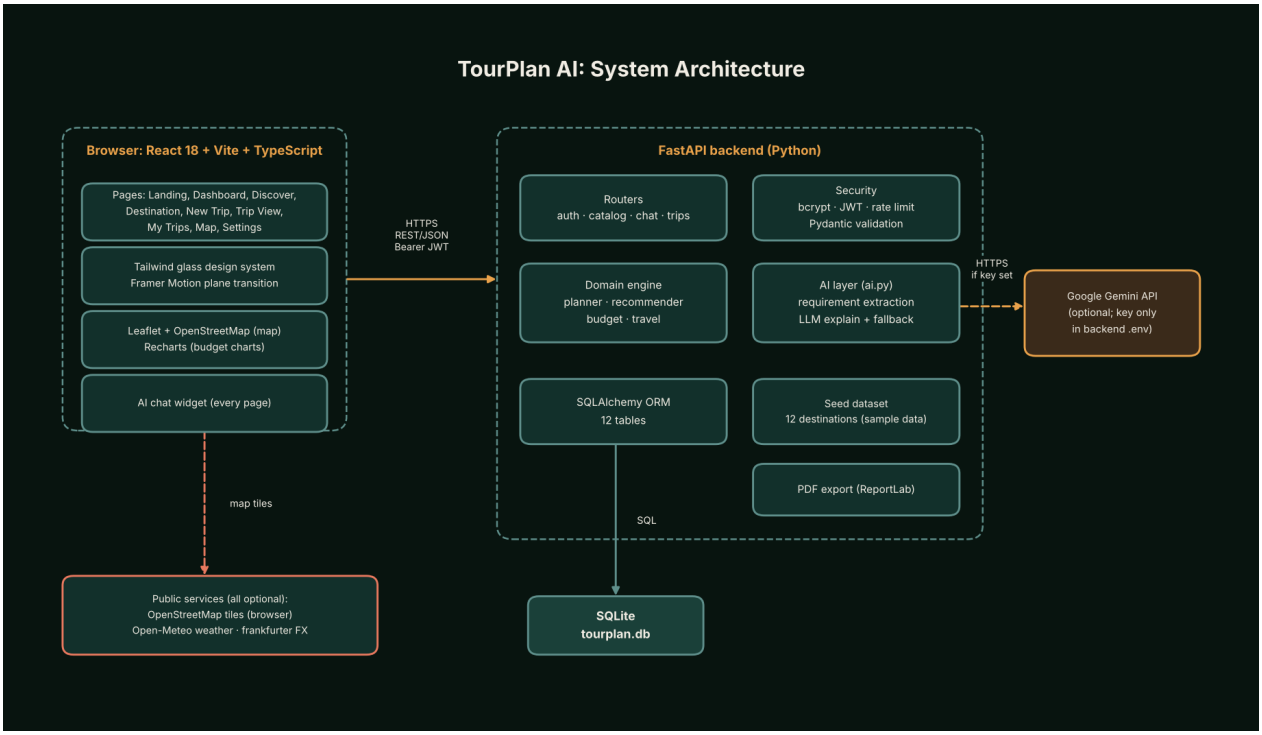


Figure 1: System architecture

11. Use Case Design

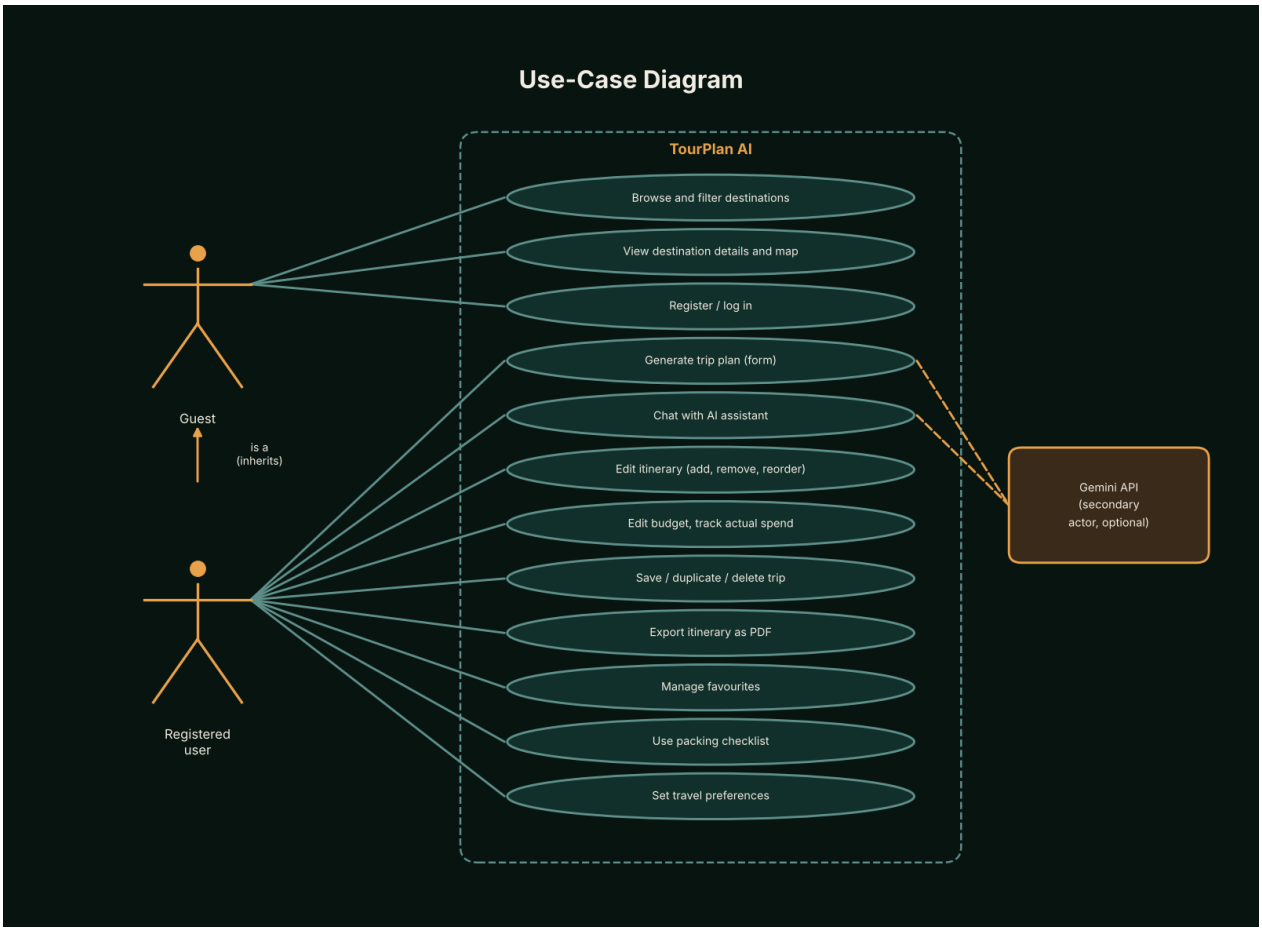


Figure 2: Use-case diagram

Use case	Actor	Description
UC-01 Register / log in	Guest	Submits name, email, password. System validates, hashes the password, returns a token. Alternate: duplicate email (409), invalid input (422), wrong credentials (401).
UC-02 Browse and filter destinations	Guest / User	Searches and filters the catalogue. Alternate: no match shows an empty state with a clear-filters action.
UC-03 Generate trip plan	User	Enters requirements. System validates, scores destinations (or uses the chosen one), builds itinerary and budget, optionally adds an LLM explanation, labels the source. Alternate: unsupported city, over-budget warning.
UC-04 Edit and save itinerary	User	Changes, adds, removes, reorders activities and saves. System validates and persists, then reloads.
UC-05 Edit budget	User	Changes estimates and actual spend; totals update instantly; saving persists them.
UC-06 Manage trips	User	Lists, searches, filters, opens, duplicates, deletes (after confirmation) trips.
UC-07 Export PDF	User	Requests the PDF; system renders itinerary and budget with an estimate disclaimer.

Use case	Actor	Description
UC-08 Chat with AI assistant	User	Describes the trip in natural language; system extracts requirements, asks for missing ones, returns a plan that can be saved as a trip.
UC-09 Manage favourites and preferences	User	Toggles favourites; edits home city, style and interests.

12. Data Flow Diagrams

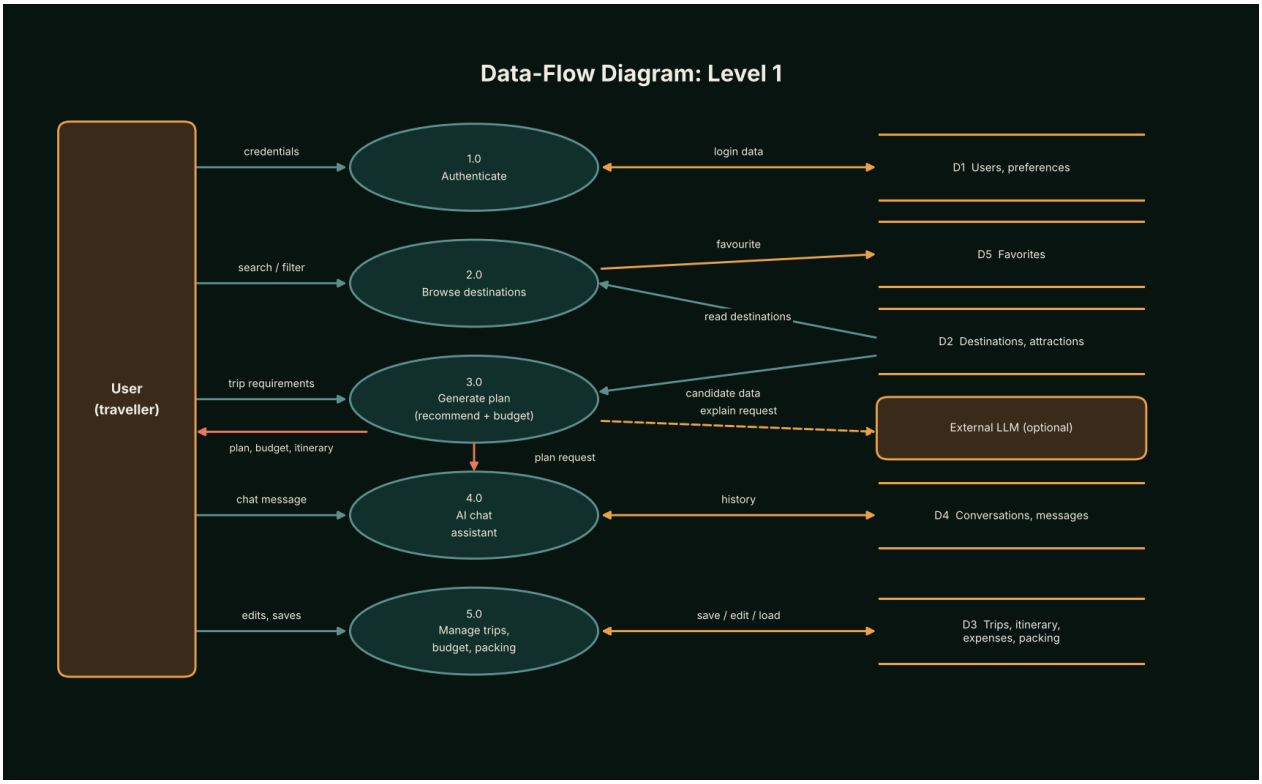


Figure 3: Data flow diagram

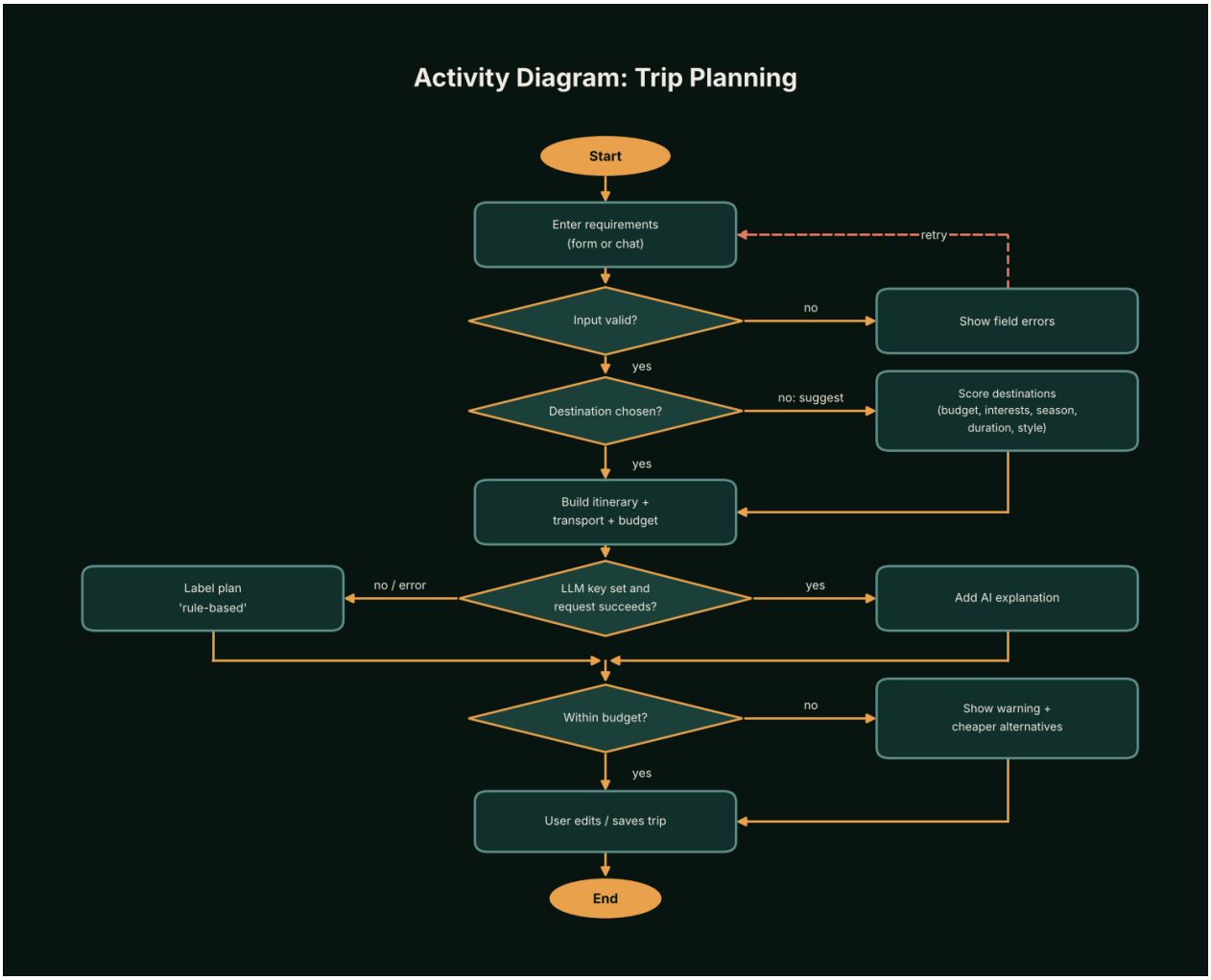


Figure 4: Activity diagram, trip planning

13. ER Diagram and Database Schema

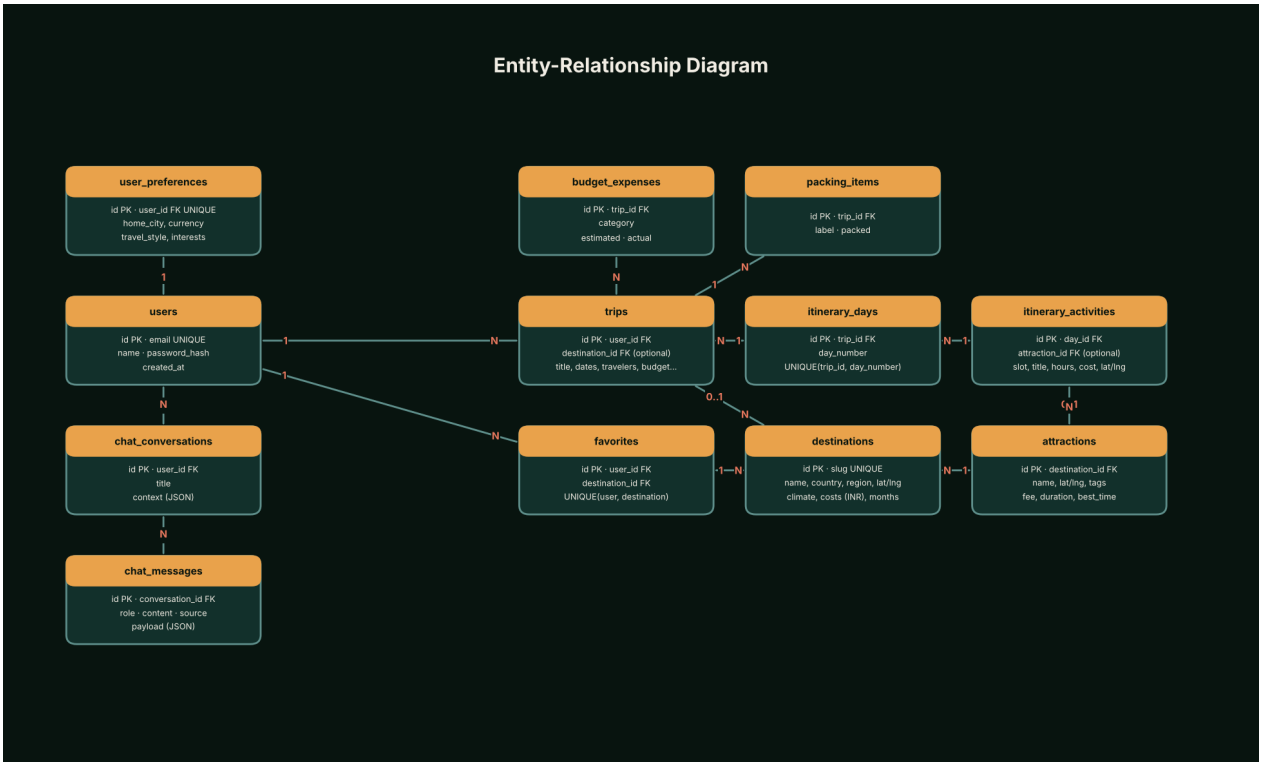


Figure 5: Entity-relationship diagram

Tables: users, user_preferences, destinations, attractions, trips, itinerary_days, itinerary_activities, budget_expenses, favorites, chat_conversations, chat_messages, packing_items. Constraints include unique (trip, day_number), unique (user, destination) favourites and enabled foreign keys. Seed data: 12 destinations and 54 attractions (sample data).

14. AI Architecture

Step 1: rule-based extraction turns chat text into structured requirements (city, days, travellers, budget, interests). Step 2: the recommender and planner build the plan from the database. Step 3: if a key is configured, Gemini is asked to return JSON explaining the plan; the reply is validated against a Pydantic model. Step 4: on any failure (no key, timeout, invalid JSON) a rule-based explanation is used and marked source = rule_based. The LLM never supplies prices, coordinates or place names.

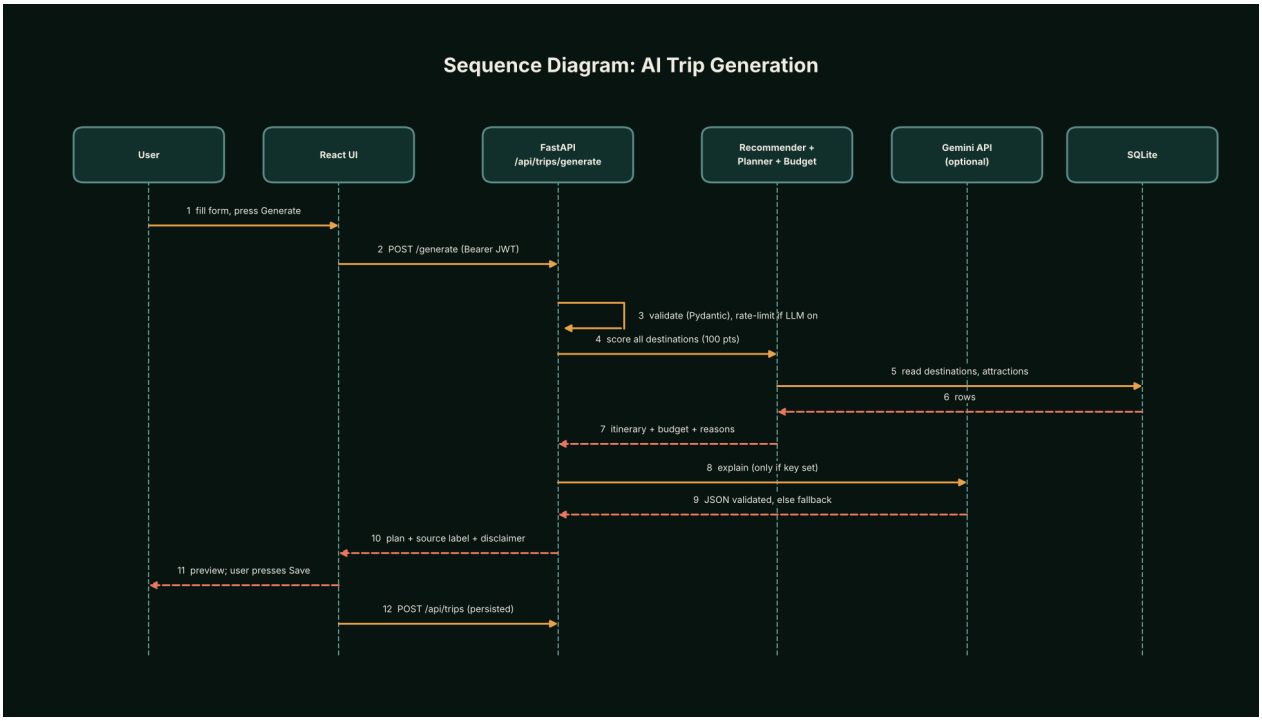


Figure 6: Sequence diagram, AI trip generation

Component	Max points
budget_fit	35
interest_match	30
season_fit	15
duration_fit	10
style_fit	10

Plus up to 8 bonus points for place-type hints. Destinations within budget rank first; every component shows a reason.

15. User Interface Design

Dark forest-green glassmorphism with amber and coral accents; responsive grid; featured destination card, stat cards, map panel, progress rings; accessibility features include labelled controls, keyboard focus, aria roles and reduced-motion handling.

• Airplane transition between routes, with a settings toggle and automatic reduced-motion bypass.

16. Screenshots

Screenshots must be captured from the running application. They are placeholders here; nothing has been faked.

[Insert screenshot: Landing page]

[Insert screenshot: Dashboard]

[Insert screenshot: Discover]

[Insert screenshot: Destination detail]

[Insert screenshot: Smart Trip Generator]

[Insert screenshot: Itinerary]

[Insert screenshot: Budget planner]

[Insert screenshot: Interactive map]

[Insert screenshot: My Trips]

[Insert screenshot: AI assistant]

[Insert screenshot: Profile / settings]

17. Module Description

• Authentication and profile

• Catalogue (destinations, favourites, stats, weather, currency)

• Trip generator (recommender, planner, transport, budget)

• Trips (CRUD, itinerary, budget, packing, PDF)

• AI assistant (extraction, rate limit, Gemini, fallback, conversations)

• Frontend pages and shared components

18. API Design

34 endpoints (extracted from the router source):

Method	Path	Handler
POST	/api/auth/register	auth.register
POST	/api/auth/login	auth.login
GET	/api/auth/me	auth.me
GET	/api/auth/preferences	auth.get_preferences
PUT	/api/auth/preferences	auth.put_preferences
GET	/api/health	catalog.health
GET	/api/cities	catalog.cities
GET	/api/destinations	catalog.list_destinations
GET	/api/destinations/{dest_id}	catalog.get_destination
GET	/api/favorites	catalog.list_favorites
POST	/api/favorites	catalog.add_favorite
DELETE	/api/favorites/{fav_id}	catalog.remove_favorite
GET	/api/stats	catalog.stats
GET	/api/weather	catalog.weather
GET	/api/currency	catalog.currency
GET	/api/ai/status	chat.ai_status
POST	/api/ai/chat	chat.chat
GET	/api/ai/conversations	chat.conversations
GET	/api/ai/conversations/{cid}	chat.conversation
DELETE	/api/ai/conversations/{cid}	chat.clear_conversation
POST	/api/trips/generate	trips.generate_trip
POST	/api/trips	trips.create_trip
GET	/api/trips	trips.list_trips
GET	/api/trips/{trip_id}	trips.get_trip
PUT	/api/trips/{trip_id}	trips.update_trip
DELETE	/api/trips/{trip_id}	trips.delete_trip
POST	/api/trips/{trip_id}/duplicate	trips.duplicate_trip
GET	/api/trips/{trip_id}/itinerary	trips.get_itinerary
PUT	/api/trips/{trip_id}/itinerary	trips.put_itinerary
GET	/api/trips/{trip_id}/budget	trips.get_budget
PUT	/api/trips/{trip_id}/budget	trips.put_budget
GET	/api/trips/{trip_id}/packing	trips.get_packing
PUT	/api/trips/{trip_id}/packing	trips.put_packing
GET	/api/trips/{trip_id}/export.pdf	trips.export_pdf

19. Budget Methodology

Categories: transport, accommodation, food, activities, local transport, shopping/misc (8% of core) and emergency buffer (10% of core). Rooms = ceil(travellers / 2). Style factors scale costs.

Worked example (identical to the unit test): 4 days, 3 travellers, stay Rs 1,000 per night, food Rs 500 per day, local Rs 200 per day, activities Rs 100 per day, transport Rs 1,000 per person round trip, fees Rs 300 per person, budget style gives a total of Rs 21,240 (Rs 7,080 per person).

All figures are estimates derived from sample data, not live prices.

20. Security Design

• bcrypt hashing; JWT with expiry; per-user authorisation (404 for foreign ids).

• In-memory AI rate limit; CORS allow-list; secrets only in backend env.

• Input validation with Pydantic; generic 500 messages.

• Limitation: JWT in localStorage; no CSRF/cookie mode; no password reset.

21. Implementation Details

Backend: routers call services and pure domain functions. Frontend: typed API client, contexts for auth, toasts and motion, `useAsync` for loading/error states. Diagrams are generated by docs/make_diagrams.py; documents by the build scripts in docs/.

22. Testing Methodology

Backend tests use pytest and FastAPI's TestClient with a temporary SQLite database, LLM disabled and live weather/currency disabled. Budget and travel logic are tested as pure functions with hand-calculated values. Frontend helpers are tested with Vitest. A manual checklist is in docs/TEST_PLAN.md.

23. Test Cases

45 automated test functions exist in the repository:

File	Test function
test_ai.py	test_extracts_demo_message
test_ai.py	test_extraction_variants
test_ai.py	test_destination_named_in_text
test_ai.py	test_asks_followup_when_details_missing
test_ai.py	test_fallback_chat_is_labelled_rule_based_and_plan_is_saveable
test_ai.py	test_conversational_refinement_and_persistence
test_ai.py	test_unrealistic_budget_is_explained_not_hidden
test_ai.py	test_conversations_are_private
test_ai.py	test_rate_limit
test_ai.py	test_ai_status_reports_fallback
test_ai.py	test_gemini_valid_json_is_parsed
test_ai.py	test_gemini_failures_raise_llmerror
test_ai.py	test_network_error_falls_back_and_is_disclosed
test_ai.py	test_llm_success_marks_source_llm
test_api.py	test_health
test_api.py	test_register_login_and_validation
test_api.py	test_demo_account_seeded
test_api.py	test_protected_routes_need_token
test_api.py	test_destinations_list_filter_detail
test_api.py	test_trip_crud_cycle
test_api.py	test_trip_validation
test_api.py	test_users_cannot_access_each_others_trips
test_api.py	test_generate_without_destination_recommends_and_is_labelled
test_api.py	test_generate_rejects_bad_city_and_dates
test_api.py	test_generated_plan_saves_and_budget_edit_recalculates
test_api.py	test_itinerary_edit_add_remove_reorder
test_api.py	test_pdf_export
test_api.py	test_favorites_flow_and_isolation
test_api.py	test_data_survives_app_restart
test_api.py	test_currency_fallback_is_labelled
test_api.py	test_weather_disabled_is_honest
test_budget.py	test_budget_hand_calculated_values
test_budget.py	test_hostel_is_cheaper_than_hotel
test_budget.py	test_rooms_round_up
test_budget.py	test_single_day_has_no_accommodation
test_budget.py	test_trip_days_inclusive_and_validation
test_budget.py	test_invalid_inputs_rejected
test_budget.py	test_budget_status_and_alternatives
test_budget.py	test_summarize_recalculates_after_manual_edit
test_budget.py	test_daily_breakdown_sums_close_to_total_minus_buffer

File	Test function
test_travel_planner.py	test_haversine_known_distance
test_travel_planner.py	test_transport_estimate_shape_and_label
test_travel_planner.py	test_short_flight_falls_back_to_train
test_travel_planner.py	test_bad_inputs
test_travel_planner.py	test_seed_coordinates_are_valid_and_unique_slugs

24. Validation and Error Handling

• Pydantic rejects bad types, ranges and date order (HTTP 422).

• Duplicate email returns 409; wrong credentials return 401; foreign trips return 404.

• Unsupported start cities and over-budget plans produce clear messages and alternatives.

• LLM, weather and currency failures degrade to labelled fallbacks.

25. Actual Test Results

No test run has been recorded in this report. Run the commands below and paste the real output here.

```
cd backend && pytest -v
cd frontend && npm run typecheck && npm test
```

[Paste pytest output here]

[Paste vitest output here]

[Number passed / failed, date, environment]

26. Limitations

• All destination data is a small hand-written sample (12 destinations, 54 attractions). Costs are indicative estimates, not live prices; coordinates of attractions are approximate.

• Transport estimates exist only for 10 start cities (Mumbai, Pune, Delhi, Bengaluru, Chennai, Hyderabad, Kolkata, Ahmedabad, Jaipur, Lucknow) and use straight-line distance with fixed per-km rates.

• The assistant's requirement extraction is rule-based (English keywords and patterns); unusual phrasing may be missed, in which case it asks follow-up questions.

• Dietary and accessibility notes are saved with a trip but are not used to filter places.

• The AI rate limiter is in memory (single process) and resets on restart.

• The JWT is kept in browser localStorage, which is exposed to cross-site-scripting if the front end were compromised; httpOnly cookies would be stronger.

• The database is created with create_all and seeded on first start; there are no migrations.

• No real destination photographs are bundled; a generated illustration is shown unless the user adds licensed images.

• Notifications and trip reminders are not implemented. Currency is INR only for trips.

• The application does not book flights, hotels or tickets.

27. Future Enhancements

• Alembic migrations and PostgreSQL for multi-user deployment.

• httpOnly cookie sessions with CSRF protection; refresh tokens.

• Live pricing integrations (rail, flight and hotel APIs) clearly separated from estimates.

• Geocoding and a larger, curated attraction dataset; real opening hours.

• Redis-backed rate limiting; streaming LLM responses.

• Collaborative trips, shared expense splitting and offline (PWA) support.

• Reminder emails or push notifications.

28. Conclusion

TourPlan AI shows how a deterministic, testable planning core can be combined with an optional language model without letting the model invent facts. The design favours transparency (visible scores, labelled estimates) over impressive but unverifiable output.

29. References

• FastAPI documentation. https://fastapi.tiangolo.com

• SQLAlchemy 2.0 documentation. https://docs.sqlalchemy.org

• Pydantic documentation. https://docs.pydantic.dev

• React documentation. https://react.dev

• Vite documentation. https://vite.dev

• Tailwind CSS documentation. https://tailwindcss.com/docs

• Leaflet documentation. https://leafletjs.com

• OpenStreetMap copyright and licence. https://www.openstreetmap.org/copyright

• Recharts documentation. https://recharts.org

• Motion (Framer Motion) documentation. https://motion.dev

• Open-Meteo weather API. https://open-meteo.com

• Frankfurter currency API. https://www.frankfurter.app

• Google Gemini API documentation. https://ai.google.dev/gemini-api/docs

• Jones, M., Bradley, J., Sakimura, N. RFC 7519: JSON Web Token (JWT). https://datatracker.ietf.org/doc/html/rfc7519

• OWASP Top 10. https://owasp.org/www-project-top-ten/

• Sommerville, I. Software Engineering (10th ed.). Pearson, 2015.

30. Appendix A: Setup Commands

```
cd backend
python -m venv .venv
# activate the venv
pip install -r requirements.txt
pytest
uvicorn app.main:app --reload

cd frontend
npm install
npm run typecheck
npm test
npm run dev
```

31. Appendix B: Sample Data Notice

All destinations, attractions, prices and durations are hand-written sample data for demonstration. Prices are indicative estimates. Coordinates are approximate. The application does not book travel.